

Aprendizaje Supervisado

Clase 20: Introducción y Árboles de Decisión

Dr. José Bavio

Departamento de Matemática
Universidad Nacional del Sur

2026

Contenido de la clase

- 1 ¿Qué es el Aprendizaje Supervisado?
- 2 Sesgo, Varianza y Overfitting
- 3 Validación Cruzada
- 4 Árboles de Decisión

¿Qué es el Aprendizaje Supervisado?

Idea central

Tenemos datos **etiquetados**: para cada observación x_i conocemos la respuesta y_i .
El objetivo es aprender una función f tal que $f(x) \approx y$ para datos nuevos.

Dicho de otra forma

Es como aprender con un **solucionario**: el modelo estudia ejemplos resueltos y luego aplica lo aprendido a ejercicios que nunca vio.

Según el tipo de y :

- $y \in \mathbb{R} \Rightarrow$ **regresión** (Unidad 3)
- $y \in \{1, \dots, K\} \Rightarrow$ **clasificación** (esta unidad)

El objetivo real: generalización

Error de entrenamiento $\neq$ error real

Memorizar los datos de entrenamiento no sirve.

Lo que importa es predecir bien datos **que el modelo nunca vio**.

Analogía

Un alumno que memoriza las respuestas del parcial anterior sin entender los conceptos va a reprobado el parcial nuevo.

El modelo que memoriza los datos de entrenamiento hace exactamente eso.

Error de generalización:

$$\text{Err} = \mathbb{E}[\ell(y, f(x))]$$

donde ℓ es la función de pérdida (ej.: pérdida 0-1 para clasificación).

Para regresión, el error de predicción descompone en:

$$\text{ECM} = \underbrace{\text{Var}[\hat{f}(x_0)]}_{\text{varianza}} + \underbrace{[\text{Sesgo}(\hat{f}(x_0))]^2}_{\text{sesgo}^2} + \underbrace{\sigma_\varepsilon^2}_{\text{irreducible}}$$

Alto sesgo (underfitting)

El modelo es **demasiado simple**.

No captura la estructura real.

Como ajustar una recta a datos claramente curvos.

Alta varianza (overfitting)

El modelo es **demasiado complejo**.

Aprende el ruido de los datos.

Como un modelo que conecta todos los puntos con una curva serpenteante.

El compromiso sesgo-varianza

Complejidad	Sesgo	Varianza
Baja	Alto	Baja
Media	Medio	Media
Alta	Bajo	Alta

En la práctica

El error de **validación** tiene forma de U en función de la complejidad. El mínimo de esa curva es el punto óptimo. El error de **entrenamiento** baja siempre: no sirve como indicador.

Tres conjuntos

- **Entrenamiento:** ajuste de los parámetros del modelo.
- **Validación:** selección de hiperparámetros y comparación de modelos.
- **Prueba (test):** evaluación final. Se toca **una única vez**.

Regla de oro

El conjunto de prueba no debe influir en ninguna decisión de modelado.
Mirarlo varias veces equivale a “hacer trampa” con ese conjunto.

Validación cruzada k -fold

Idea: dividir el conjunto de entrenamiento en k partes iguales (*folds*).

- 1 Dividir en k folds.
- 2 Para $j = 1, \dots, k$: entrenar con los otros $k - 1$ folds, evaluar en el fold j .
- 3 Promediar los k errores:

$$CV_k = \frac{1}{k} \sum_{j=1}^k \text{Err}_j$$

Coloquialmente

Es como rendir el mismo examen 5 veces con distintos grupos de alumnos para tener una estimación más confiable de cuánto aprendió el modelo.

Valores típicos: $k = 5$ o $k = 10$.

Caso extremo: $k = n$ (*leave-one-out*, LOOCV).

Problema con clases desbalanceadas

Si el 95% de los datos son clase A y solo el 5% clase B, un fold podría no tener ningún ejemplo de clase B.

Solución: CV estratificada

Cada fold mantiene la misma **proporción de clases** que el conjunto original.
Es lo que hace `StratifiedKFold` en `scikit-learn`. Siempre usarla en clasificación.

¿Qué es un árbol de decisión?

Idea

Particionar el espacio de features con reglas del tipo

$$“x_j \leq t \quad \text{¿sí o no?”}$$

hasta llegar a regiones lo más **puras** posible.

Analogía cotidiana

Es exactamente el juego del **Akinator** o el “animal, vegetal o mineral”: preguntas binarias sucesivas que van acotando las posibilidades hasta llegar a una respuesta.

Estructura:

- **Nodo raíz:** todos los datos.
- **Nodos internos:** cada uno hace una pregunta $x_j \leq t$.
- **Hojas:** regiones terminales que asignan una clase.

Índice de Gini para un nodo con proporciones de clase $p_1, \dots, p_K$:

$$G = 1 - \sum_{k=1}^K p_k^2$$

- $G = 0$: nodo **puro** (todas las observaciones son de la misma clase). Ideal.
- G máximo: clases perfectamente mezcladas. Nodo inútil.

Intuición

Gini mide la probabilidad de equivocarse si clasificamos al azar según las proporciones del nodo. Un nodo puro tiene probabilidad de error 0, o sea, $G = 0$.

Criterios de partición: Entropía

Entropía (ganancia de información):

$$H = - \sum_{k=1}^K p_k \log_2(p_k)$$

La **ganancia de información** de un split es la caída de entropía:

$$\Delta H = H(\text{padre}) - \frac{n_L}{n} H(\text{izq.}) - \frac{n_R}{n} H(\text{der.})$$

¿Gini o entropía?

En la práctica producen resultados casi idénticos. Gini es más rápido (no requiere logaritmos). scikit-learn usa Gini por defecto.

CART (*Classification and Regression Trees*) construye el árbol de manera *greedy*:

- 1 Partir de todos los datos en el nodo raíz.
- 2 Para cada feature j y cada umbral t : calcular la impureza del split.
- 3 Elegir el par (j^*, t^*) que minimiza la impureza total.
- 4 Repetir en cada subárbol (recursión).
- 5 Parar cuando se cumple un criterio de parada.

Greedy $\neq$ óptimo global

El árbol construido por CART no es necesariamente el mejor árbol posible. Elegir el mejor split en cada nodo no garantiza el mejor árbol completo. Es como elegir siempre la jugada que parece mejor en ajedrez sin pensar en las próximas jugadas.

Cuándo dejar de crecer el árbol

- `max_depth`: profundidad máxima del árbol.
- `min_samples_split`: mínimo de muestras para dividir un nodo.
- `min_samples_leaf`: mínimo de muestras en una hoja.
- `max_leaf_nodes`: máximo de hojas totales.

Intuición

Sin restricciones, el árbol crece hasta que cada hoja tiene un solo dato: accuracy 100% en entrenamiento, pero generaliza pésimo. Los criterios de parada son la “poda preventiva”.

Poda por complejidad de costo

Otra estrategia: dejar crecer el árbol al máximo y luego **podarlo**.

Se busca el subárbol T que minimiza:

$$C_{\alpha}(T) = \underbrace{\text{error de clasificación}}_{\text{en entrenamiento}} + \alpha \cdot \underbrace{|T|}_{\text{n}^{\circ} \text{ de hojas}}$$

- $\alpha = 0$: árbol completo, sin penalización.
- α grande: árbol muy pequeño (solo la raíz si $\alpha \rightarrow \infty$).
- El α óptimo se elige por validación cruzada.

Analogía

Es como la regulación L_1/L_2 en regresión, pero para árboles: penalizamos la complejidad del modelo para evitar el sobreajuste.

Ventajas y limitaciones de los árboles

Ventajas

- Muy interpretables:
“Si $x_1 > 3$ y $x_2 \leq 7$, entonces clase A”
- No requieren normalización
- Manejan variables mixtas
- Capturan interacciones

Limitaciones

- Alta varianza: cambios pequeños en los datos generan árboles muy distintos
- Fronteras de decisión solo axiales (paralelas a los ejes)
- Tendencia al sobreajuste

Por eso se usan como base de ensambles

Un árbol solo es inestable. Muchos árboles juntos son robustos.
Eso es exactamente Random Forest (próxima clase).

- El **aprendizaje supervisado** aprende de ejemplos etiquetados para predecir datos nuevos.
- El objetivo es la **generalización**, no el ajuste perfecto al entrenamiento.
- El compromiso **sesgo-varianza** explica el underfitting y el overfitting.
- La **validación cruzada k -fold** estima el error de generalización.
- Los **árboles de decisión** particionan el espacio con reglas binarias usando Gini o entropía.
- El algoritmo **CART** es greedy; la poda controla la complejidad.

Próxima clase: Random Forest — cómo convertir árboles inestables en un modelo robusto.